

Spécification du Système Autonome MK4

Architecture logicielle embarquée et pipeline de traitement temps réel

Robin Forestier

Août 2026

Table des matières

1	Introduction et Objectifs	2
1.1	Exigences fonctionnelles	2
2	Modèle Mathématique et Filtrage	2
3	Implémentation Logicielle	2
4	Évaluation des Performances	4
5	Conclusion et Perspectives	4

1 Introduction et Objectifs

Le projet **MK4** a pour mission de fournir une plateforme haute performance pour l'acquisition et le traitement temps réel de données télémétriques. Ce document détaille les spécifications d'ingénierie logicielle, les contraintes de latence et les algorithmes de filtrage embarqués.

Note Ce document est une spécification technique de référence soumise à révision périodique.

1.1 Exigences fonctionnelles

Les exigences principales sont synthétisées ci-dessous :

50

- Traitement déterministe avec une gigue (*jitter*) inférieure à μ s.
- Prise en charge des protocoles industriels standard¹.
- Taux de disponibilité nominal garanti : **99.99%**.
- ☒ Initialisation du bus CAN-FD
- ☒ Étalonnage des capteurs inertiels (IMU)
- ☐ Déploiement du module de communication satellite

2 Modèle Mathématique et Filtrage

L'estimation d'état s'appuie sur une formulation discrète du filtre de Kalman étendu (EKF). Le vecteur d'état à l'instant k est noté $x_k \in \mathbb{R}^n$.

L'équation de prédiction de l'état s'écrit :

$$x_{\{k|k-1\}} = F_k x_{\{k-1|k-1\}} + B_k u_k$$

La mise à jour de la matrice de covariance de l'erreur est donnée par :

$$P_{\{k|k-1\}} = F_k P_{\{k-1|k-1\}} F_k^T + Q_k$$

Attention Attention : la matrice de bruit Q_k doit être symétrique définie positive pour garantir la convergence numérique du filtre.

3 Implémentation Logicielle

Le cœur de calcul est implémenté en langage **Rust** pour assurer l'absence totale de ramasse-miettes (*garbage collector*) et garantir la sûreté mémoire sans surcoût d'exécution.

¹Protocoles supportés : CAN-FD, SPI haut débit, UART asynchrone et Ethernet TSN.

```
pipeline.rs

1      use std::time::Instant;
2
3      pub struct TelemetryPipeline {
4          buffer_size: usize,
5          is_calibrated: bool,
6      }
7
8      impl TelemetryPipeline {
9      pub fn process_packet(&mut self, payload: &[u8]) -> Result<Status, Error>
10         {
11             let start = Instant::now();
12             // Traitement vectorisé SIMD
13             let state = self.decode_and_filter(payload)?;
14
15             println!("Cycle time: {:?}", start.elapsed());
16             Ok(state)
17         }
18     }
```

Liste 1. – Pipeline de décodage et filtrage temps réel

Astuce Conseil d'optimisation : Activez les drapeaux de compilation `-C target-cpu=native` pour tirer parti des instructions vectorielles AVX-512 sur cible `x86_64`.

4 Évaluation des Performances

Les mesures de latence ont été réalisées sur un banc d'essai matériel composé d'un microcontrôleur STM32H7 cadencé à 480 MHz.

Module	Fréquence	Latence moyenne	Consommation	Statut
Décodage CAN	1 kHz	$12 \mu s$	45 mW	Nominal
Filtrage EKF	500 Hz	$38 \mu s$	110 mW	Nominal
Chiffrement AES	100 Hz	$85 \mu s$	65 mW	En cours

Tableau 1. – Synthèse des métriques d'exécution sur cible embarquée

Comme illustré dans le Tableau 1, la latence cumulée respecte strictement la contrainte budgétaire des $50 \mu s$ définie dans la Chapitre 1.

5 Conclusion et Perspectives

L'architecture validée en Chapitre 3 répond à l'ensemble des contraintes de performance. Les prochaines étapes porteront sur l'intégration du sous-système de redondance matérielle et la qualification environnementale.